

Vidu X-ray project

March 29, 2026

1 Setup

```
[37]: import os
      from pathlib import Path

      import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      import seaborn as sns
      from PIL import Image

      import torch
      import torch.nn as nn
      import torch.nn.functional as F
      from torchvision import transforms, models
      from torch.utils.data import Dataset, DataLoader
      from sklearn.model_selection import train_test_split
```

```
[3]: pd.options.display.float_format = '{:.2f}'.format
```

Configurations

```
[4]: KAGGLE_DIR = Path('/kaggle/input/competitions/26-t-1-dl-gen-ainppe-1')
      IS_KAGGLE = KAGGLE_DIR.exists()

      DATA_DIR = KAGGLE_DIR if IS_KAGGLE else Path('./data')
      IMG_DIR = DATA_DIR / 'images'
      DEBUG = not IS_KAGGLE
      NUM_WORKERS = 2 if IS_KAGGLE else 0

      print(f'{IS_KAGGLE=}')
      print(f'{DEBUG=}')
      print(f'{DATA_DIR=}')

IS_KAGGLE=False
DEBUG=True
DATA_DIR=WindowsPath('data')
```

Data loading

```
[5]: train_df = pd.read_csv(DATA_DIR / 'train.csv')
test_df = pd.read_csv(DATA_DIR / 'test.csv')

print(f'Shape of train: {train_df.shape}')
print(f'Shape of test: {test_df.shape}')

print('\nPreview of train_df:')
train_df.head(3)
```

Shape of train: (51043, 21)

Shape of test: (17015, 1)

Preview of train_df:

```
[5]:
```

	id	Atelectasis	Cardiomegaly	\
0	977df638b5294072ac81c369e2d9ecd0.png	0	0	
1	4f6dd5e39cd548df904b7319b13a40c5.png	0	0	
2	8772d25762484c2aa3f3e124d2ebcb30.png	1	0	

	Consolidation	Edema	Effusion	Emphysema	Fibrosis	Hernia	Infiltration	\
0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	

	...	Nodule	Pleural_Thickening	Pneumonia	Pneumothorax	Pneumoperitoneum	\
0	...	0	0	0	0	0	
1	...	0	0	0	0	0	
2	...	0	0	0	0	0	

	Pneumomediastinum	Subcutaneous	Emphysema	Tortuous Aorta	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	

	Calcification of the Aorta	No Finding
0	0	1
1	0	1
2	0	0

[3 rows x 21 columns]

Ensure all files are present

```
[6]: files_req = set(train_df['id']) | set(test_df['id'])
files_in_img_dir = set(p.name for p in IMG_DIR.glob('*.png'))

if len(files_req - files_in_img_dir) == 0:
    print('All image files are present ')
```

All image files are present

2 Exploratory data analysis

Take a look at first image file

```
[7]: img_path = IMG_DIR / train_df.loc[0, 'id']
img = Image.open(img_path)
img_arr = np.array(img)

print(f'Size (W, H): {img.size}')
print(f'Mode: {img.mode}')
print(f'Image as np.array (H, W, C): {img_arr.shape}')

_ = plt.imshow(img)
_ = plt.axis('off')
```

Size (W, H): (384, 384)

Mode: RGB

Image as np.array (H, W, C): (384, 384, 3)



Check if all channels are identical

The image is actually a grayscale image. Does same grayscale values are repeated in three channels?

```
[8]: r = img_arr[:, :, 0]
      g = img_arr[:, :, 1]
      b = img_arr[:, :, 2]

      np.array_equal(r, g) and np.array_equal(g, b)
```

[8]: True

Looking at all disease classes

```
[9]: disease_classes = train_df.columns[1:].tolist()
      print(len(disease_classes))
      print(disease_classes)
```

20

```
['Atelectasis', 'Cardiomegaly', 'Consolidation', 'Edema', 'Effusion',
'Emphysema', 'Fibrosis', 'Hernia', 'Infiltration', 'Mass', 'Nodule',
'Pleural_Thickening', 'Pneumonia', 'Pneumothorax', 'Pneumoperitoneum',
'Pneumomediastinum', 'Subcutaneous Emphysema', 'Tortuous Aorta', 'Calcification
of the Aorta', 'No Finding']
```

How many samples per class in our train data?

```
[10]: class_counts = train_df[disease_classes].sum(axis=0)
      class_counts = class_counts.sort_values(ascending=False)
      class_counts_pct = class_counts / len(train_df) * 100

      pd.DataFrame({
          'Count': class_counts,
          '%': class_counts_pct,
      })
```

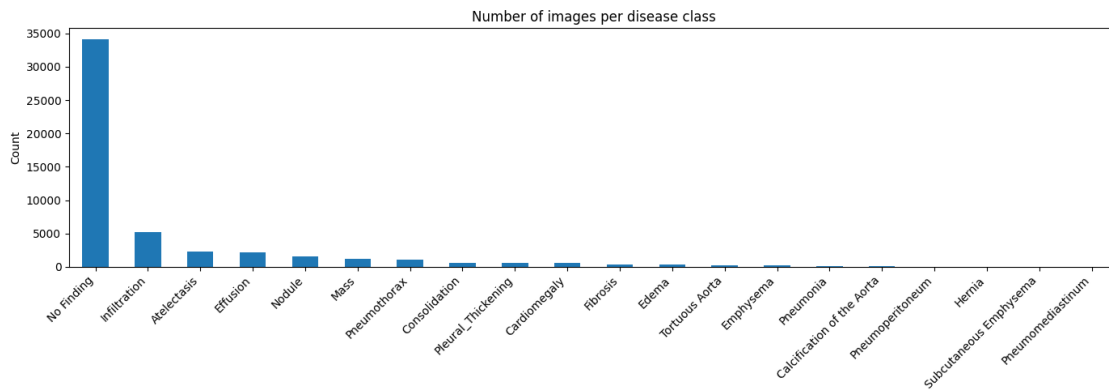
```
[10]:
```

	Count	%
No Finding	34079	66.77
Infiltration	5206	10.20
Atelectasis	2351	4.61
Effusion	2156	4.22
Nodule	1527	2.99
Mass	1249	2.45
Pneumothorax	1114	2.18
Consolidation	651	1.28
Pleural_Thickening	608	1.19
Cardiomegaly	600	1.18
Fibrosis	389	0.76
Edema	326	0.64
Tortuous Aorta	254	0.50
Emphysema	172	0.34
Pneumonia	160	0.31
Calcification of the Aorta	91	0.18

Pneumoperitoneum	44	0.09
Hernia	37	0.07
Subcutaneous Emphysema	24	0.05
Pneumomediastinum	5	0.01

We can see that our data is extremely imbalanced. We need to take care of this during model training and evaluation.

```
[11]: _ = plt.figure(figsize=(14, 5))
_ = class_counts.plot(kind='bar')
_ = plt.title('Number of images per disease class')
_ = plt.ylabel('Count')
_ = plt.xticks(rotation=45, ha='right')
_ = plt.tight_layout()
```



3 Dataset & DataLoader

```
[12]: # Training: add some random flips/rotations to help the model generalize
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),          # resize to standard size
    transforms.RandomHorizontalFlip(),      # randomly flip left-right
    transforms.RandomRotation(10),          # randomly rotate ±10 degrees
    transforms.ToTensor(),                  # convert image to numbers
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225]) # standard ImageNet
])

# Validation/Test: no random augmentation, just resize and normalize
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
```

```
[0.229, 0.224, 0.225])  
])
```

```
[13]: class ChestXrayDataset(Dataset):  
    def __init__(self, dataframe, img_dir, transform):  
        self.df = dataframe.reset_index(drop=True)  
        self.img_dir = img_dir  
        self.transform = transform  
        self.disease_classes = dataframe.columns[1:].tolist()  
  
    def __len__(self):  
        # how many images total  
        return len(self.df)  
  
    def __getitem__(self, idx):  
        row = self.df.iloc[idx]  
        img_path = self.img_dir / row.iloc[0]  
  
        # open the image  
        img = Image.open(img_path).convert('RGB')  
        img = self.transform(img)  
  
        # find which column has the "1" → that's the label  
        label_values = row[self.disease_classes].values.astype(float)  
        label = torch.tensor(label_values).argmax().long()  
  
        return img, label
```

Train test splitting

```
[14]: # Get class index for each image (used for stratified split)  
label_indices = train_df[disease_classes].values.argmax(axis=1)  
  
train_split_df, val_split_df = train_test_split(  
    train_df,  
    test_size=0.20,  
    random_state=42,  
    stratify=label_indices # keep class ratios same in both splits  
)  
  
print("Train size:", len(train_split_df))  
print("Val size: ", len(val_split_df))
```

Train size: 40834

Val size: 10209

Create Dataset objects and DataLoaders

```
[15]: train_dataset = ChestXrayDataset(train_split_df, IMG_DIR, train_transform)
      val_dataset = ChestXrayDataset(val_split_df, IMG_DIR, val_transform)
```

```
print('# Samples in train:', len(train_dataset))
print('# Samples in val:', len(val_dataset))
```

Samples in train: 40834

Samples in val: 10209

Looking at first sample in train dataset

```
[16]: img, label = train_dataset[0]
      print(type(img))
      print(img.shape)  # (C, H, W)
      print(label)
```

```
<class 'torch.Tensor'>
torch.Size([3, 224, 224])
tensor(19)
```

```
[35]: BATCH_SIZE = 32

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS
)

val_loader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS
)

print("Train batches:", len(train_loader))
print("Val batches: ", len(val_loader))
```

Train batches: 1277

Val batches: 320

Looking at the first batch of train loader

```
[33]: features, labels = next(iter(train_loader))
      print(features.shape)  # (BATCH_SIZE, C, H, W)
      print(labels.shape)  # (BATCH_SIZE)
```

```
torch.Size([32, 3, 224, 224])
torch.Size([32])
```

4 Model building

```
[56]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
      print("Using device:", device)
```

Using device: cpu

```
[63]: weights = models.DenseNet121_Weights.DEFAULT
      model = models.densenet121(weights=weights)
```

```
[64]: # Looking at the meta properties
      for key, val in weights.meta.items():
          print(f'{key}: {val}')
```

min_size: (29, 29)

categories: ['tench', 'goldfish', 'great white shark', 'tiger shark', 'hammerhead', 'electric ray', 'stingray', 'cock', 'hen', 'ostrich', 'brambling', 'goldfinch', 'house finch', 'junco', 'indigo bunting', 'robin', 'bulbul', 'jay', 'magpie', 'chickadee', 'water ouzel', 'kite', 'bald eagle', 'vulture', 'great grey owl', 'European fire salamander', 'common newt', 'eft', 'spotted salamander', 'axolotl', 'bullfrog', 'tree frog', 'tailed frog', 'loggerhead', 'leatherback turtle', 'mud turtle', 'terrapin', 'box turtle', 'banded gecko', 'common iguana', 'American chameleon', 'whiptail', 'agama', 'frilled lizard', 'alligator lizard', 'Gila monster', 'green lizard', 'African chameleon', 'Komodo dragon', 'African crocodile', 'American alligator', 'triceratops', 'thunder snake', 'ringneck snake', 'hognose snake', 'green snake', 'king snake', 'garter snake', 'water snake', 'vine snake', 'night snake', 'boa constrictor', 'rock python', 'Indian cobra', 'green mamba', 'sea snake', 'horned viper', 'diamondback', 'sidewinder', 'trilobite', 'harvestman', 'scorpion', 'black and gold garden spider', 'barn spider', 'garden spider', 'black widow', 'tarantula', 'wolf spider', 'tick', 'centipede', 'black grouse', 'ptarmigan', 'ruffed grouse', 'prairie chicken', 'peacock', 'quail', 'partridge', 'African grey', 'macaw', 'sulphur-crested cockatoo', 'lorikeet', 'coucal', 'bee eater', 'hornbill', 'hummingbird', 'jacamar', 'toucan', 'drake', 'red-breasted merganser', 'goose', 'black swan', 'tusker', 'echidna', 'platypus', 'wallaby', 'koala', 'wombat', 'jellyfish', 'sea anemone', 'brain coral', 'flatworm', 'nematode', 'conch', 'snail', 'slug', 'sea slug', 'chiton', 'chambered nautilus', 'Dungeness crab', 'rock crab', 'fiddler crab', 'king crab', 'American lobster', 'spiny lobster', 'crayfish', 'hermit crab', 'isopod', 'white stork', 'black stork', 'spoonbill', 'flamingo', 'little blue heron', 'American egret', 'bittern', 'crane bird', 'limpkin', 'European gallinule', 'American coot', 'bustard', 'ruddy turnstone', 'red-backed sandpiper', 'redshank', 'dowitcher', 'oystercatcher', 'pelican', 'king penguin', 'albatross', 'grey whale', 'killer whale', 'dugong', 'sea lion', 'Chihuahua', 'Japanese spaniel', 'Maltese dog', 'Pekinese', 'Shih-Tzu', 'Blenheim spaniel', 'papillon', 'toy terrier', 'Rhodesian ridgeback', 'Afghan hound', 'basset', 'beagle', 'bloodhound', 'bluetick', 'black-and-tan coonhound', 'Walker hound', 'English foxhound', 'redbone', 'borzoi', 'Irish wolfhound', 'Italian greyhound', 'whippet', 'Ibizan

hound', 'Norwegian elkhound', 'otterhound', 'Saluki', 'Scottish deerhound',
'Weimaraner', 'Staffordshire bullterrier', 'American Staffordshire terrier',
'Bedlington terrier', 'Border terrier', 'Kerry blue terrier', 'Irish terrier',
'Norfolk terrier', 'Norwich terrier', 'Yorkshire terrier', 'wire-haired fox
terrier', 'Lakeland terrier', 'Sealyham terrier', 'Airedale', 'cairn',
'Australian terrier', 'Dandie Dinmont', 'Boston bull', 'miniature schnauzer',
'giant schnauzer', 'standard schnauzer', 'Scotch terrier', 'Tibetan terrier',
'silky terrier', 'soft-coated wheaten terrier', 'West Highland white terrier',
'Lhasa', 'flat-coated retriever', 'curly-coated retriever', 'golden retriever',
'Labrador retriever', 'Chesapeake Bay retriever', 'German short-haired pointer',
'vizsla', 'English setter', 'Irish setter', 'Gordon setter', 'Brittany spaniel',
'clumber', 'English springer', 'Welsh springer spaniel', 'cocker spaniel',
'Sussex spaniel', 'Irish water spaniel', 'kuvasz', 'schipperke', 'groenendael',
'malinois', 'briard', 'kelpie', 'komondor', 'Old English sheepdog', 'Shetland
sheepdog', 'collie', 'Border collie', 'Bouvier des Flandres', 'Rottweiler',
'German shepherd', 'Doberman', 'miniature pinscher', 'Greater Swiss Mountain
dog', 'Bernese mountain dog', 'Appenzeller', 'EntleBucher', 'boxer', 'bull
mastiff', 'Tibetan mastiff', 'French bulldog', 'Great Dane', 'Saint Bernard',
'Eskimo dog', 'malamute', 'Siberian husky', 'dalmatian', 'affenpinscher',
'basenji', 'pug', 'Leonberg', 'Newfoundland', 'Great Pyrenees', 'Samoyed',
'Pomeranian', 'chow', 'keeshond', 'Brabancon griffon', 'Pembroke', 'Cardigan',
'toy poodle', 'miniature poodle', 'standard poodle', 'Mexican hairless', 'timber
wolf', 'white wolf', 'red wolf', 'coyote', 'dingo', 'dhole', 'African hunting
dog', 'hyena', 'red fox', 'kit fox', 'Arctic fox', 'grey fox', 'tabby', 'tiger
cat', 'Persian cat', 'Siamese cat', 'Egyptian cat', 'cougar', 'lynx', 'leopard',
'snow leopard', 'jaguar', 'lion', 'tiger', 'cheetah', 'brown bear', 'American
black bear', 'ice bear', 'sloth bear', 'mongoose', 'meerkat', 'tiger beetle',
'ladybug', 'ground beetle', 'long-horned beetle', 'leaf beetle', 'dung beetle',
'rhinoceros beetle', 'weevil', 'fly', 'bee', 'ant', 'grasshopper', 'cricket',
'walking stick', 'cockroach', 'mantis', 'cicada', 'leafhopper', 'lacewing',
'dragonfly', 'damselfly', 'admiral', 'ringlet', 'monarch', 'cabbage butterfly',
'sulphur butterfly', 'lycaenid', 'starfish', 'sea urchin', 'sea cucumber', 'wood
rabbit', 'hare', 'Angora', 'hamster', 'porcupine', 'fox squirrel', 'marmot',
'beaver', 'guinea pig', 'sorrel', 'zebra', 'hog', 'wild boar', 'warthog',
'hippopotamus', 'ox', 'water buffalo', 'bison', 'ram', 'bighorn', 'ibex',
'hartebeest', 'impala', 'gazelle', 'Arabian camel', 'llama', 'weasel', 'mink',
'polecat', 'black-footed ferret', 'otter', 'skunk', 'badger', 'armadillo',
'three-toed sloth', 'orangutan', 'gorilla', 'chimpanzee', 'gibbon', 'siamang',
'guenon', 'patas', 'baboon', 'macaque', 'langur', 'colobus', 'proboscis monkey',
'marmoset', 'capuchin', 'howler monkey', 'titi', 'spider monkey', 'squirrel
monkey', 'Madagascar cat', 'indri', 'Indian elephant', 'African elephant',
'lesser panda', 'giant panda', 'barracouta', 'eel', 'coho', 'rock beauty',
'anemone fish', 'sturgeon', 'gar', 'lionfish', 'puffer', 'abacus', 'abaya',
'academic gown', 'accordion', 'acoustic guitar', 'aircraft carrier', 'airliner',
'airship', 'altar', 'ambulance', 'amphibian', 'analog clock', 'apiary', 'apron',
'ashcan', 'assault rifle', 'backpack', 'bakery', 'balance beam', 'balloon',
'ballpoint', 'Band Aid', 'banjo', 'bannister', 'barbell', 'barber chair',
'barbershop', 'barn', 'barometer', 'barrel', 'barrow', 'baseball', 'basketball',

'bassinet', 'bassoon', 'bathing cap', 'bath towel', 'bathtub', 'beach wagon',
'beacon', 'beaker', 'bearskin', 'beer bottle', 'beer glass', 'bell cote', 'bib',
'bicycle-built-for-two', 'bikini', 'binder', 'binoculars', 'birdhouse',
'boathouse', 'bobsled', 'bolo tie', 'bonnet', 'bookcase', 'bookshop',
'bottlecap', 'bow', 'bow tie', 'brass', 'brassiere', 'breakwater',
'breastplate', 'broom', 'bucket', 'buckle', 'bulletproof vest', 'bullet train',
'butcher shop', 'cab', 'caldron', 'candle', 'cannon', 'canoe', 'can opener',
'cardigan', 'car mirror', 'carousel', "carpenter's kit", 'carton', 'car wheel',
'cash machine', 'cassette', 'cassette player', 'castle', 'catamaran', 'CD
player', 'cello', 'cellular telephone', 'chain', 'chainlink fence', 'chain
mail', 'chain saw', 'chest', 'chiffonier', 'chime', 'china cabinet', 'Christmas
stocking', 'church', 'cinema', 'cleaver', 'cliff dwelling', 'cloak', 'clog',
'cocktail shaker', 'coffee mug', 'coffeepot', 'coil', 'combination lock',
'computer keyboard', 'confectionery', 'container ship', 'convertible',
'corkscrew', 'cornet', 'cowboy boot', 'cowboy hat', 'cradle', 'crane', 'crash
helmet', 'crate', 'crib', 'Crock Pot', 'croquet ball', 'crutch', 'cuirass',
'dam', 'desk', 'desktop computer', 'dial telephone', 'diaper', 'digital clock',
'digital watch', 'dining table', 'dishrag', 'dishwasher', 'disk brake', 'dock',
'dogsled', 'dome', 'doormat', 'drilling platform', 'drum', 'drumstick',
'dumbbell', 'Dutch oven', 'electric fan', 'electric guitar', 'electric
locomotive', 'entertainment center', 'envelope', 'espresso maker', 'face
powder', 'feather boa', 'file', 'fireboat', 'fire engine', 'fire screen',
'flagpole', 'flute', 'folding chair', 'football helmet', 'forklift', 'fountain',
'fountain pen', 'four-poster', 'freight car', 'French horn', 'frying pan', 'fur
coat', 'garbage truck', 'gasmask', 'gas pump', 'goblet', 'go-kart', 'golf ball',
'golfcart', 'gondola', 'gong', 'gown', 'grand piano', 'greenhouse', 'grille',
'grocery store', 'guillotine', 'hair slide', 'hair spray', 'half track',
'hammer', 'hamper', 'hand blower', 'hand-held computer', 'handkerchief', 'hard
disc', 'harmonica', 'harp', 'harvester', 'hatchet', 'holster', 'home theater',
'honeycomb', 'hook', 'hoopskirt', 'horizontal bar', 'horse cart', 'hourglass',
'iPod', 'iron', "jack-o'-lantern", 'jean', 'jeep', 'jersey', 'jigsaw puzzle',
'jinrikisha', 'joystick', 'kimono', 'knee pad', 'knot', 'lab coat', 'ladle',
'lampshade', 'laptop', 'lawn mower', 'lens cap', 'letter opener', 'library',
'lifeboat', 'lighter', 'limousine', 'liner', 'lipstick', 'Loafer', 'lotion',
'loudspeaker', 'loupe', 'lumbermill', 'magnetic compass', 'mailbag', 'mailbox',
'maillot', 'maillot tank suit', 'manhole cover', 'maraca', 'marimba', 'mask',
'matchstick', 'maypole', 'maze', 'measuring cup', 'medicine chest', 'megalith',
'microphone', 'microwave', 'military uniform', 'milk can', 'minibus',
'miniskirt', 'minivan', 'missile', 'mitten', 'mixing bowl', 'mobile home',
'Model T', 'modem', 'monastery', 'monitor', 'moped', 'mortar', 'mortarboard',
'mosque', 'mosquito net', 'motor scooter', 'mountain bike', 'mountain tent',
'mouse', 'mousetrap', 'moving van', 'muzzle', 'nail', 'neck brace', 'necklace',
'nipple', 'notebook', 'obelisk', 'oboe', 'ocarina', 'odometer', 'oil filter',
'organ', 'oscilloscope', 'overskirt', 'oxcart', 'oxygen mask', 'packet',
'paddle', 'paddlewheel', 'padlock', 'paintbrush', 'pajama', 'palace', 'panpipe',
'paper towel', 'parachute', 'parallel bars', 'park bench', 'parking meter',
'passenger car', 'patio', 'pay-phone', 'pedestal', 'pencil box', 'pencil
sharpener', 'perfume', 'Petri dish', 'photocopier', 'pick', 'pickelhaube',

'picket fence', 'pickup', 'pier', 'piggy bank', 'pill bottle', 'pillow', 'ping-pong ball', 'pinwheel', 'pirate', 'pitcher', 'plane', 'planetarium', 'plastic bag', 'plate rack', 'plow', 'plunger', 'Polaroid camera', 'pole', 'police van', 'poncho', 'pool table', 'pop bottle', 'pot', 'potter's wheel', 'power drill', 'prayer rug', 'printer', 'prison', 'projectile', 'projector', 'puck', 'punching bag', 'purse', 'quill', 'quilt', 'racer', 'racket', 'radiator', 'radio', 'radio telescope', 'rain barrel', 'recreational vehicle', 'reel', 'reflex camera', 'refrigerator', 'remote control', 'restaurant', 'revolver', 'rifle', 'rocking chair', 'rotisserie', 'rubber eraser', 'rugby ball', 'rule', 'running shoe', 'safe', 'safety pin', 'saltshaker', 'sandal', 'sarong', 'sax', 'scabbard', 'scale', 'school bus', 'schooner', 'scoreboard', 'screen', 'screw', 'screwdriver', 'seat belt', 'sewing machine', 'shield', 'shoe shop', 'shoji', 'shopping basket', 'shopping cart', 'shovel', 'shower cap', 'shower curtain', 'ski', 'ski mask', 'sleeping bag', 'slide rule', 'sliding door', 'slot', 'snorkel', 'snowmobile', 'snowplow', 'soap dispenser', 'soccer ball', 'sock', 'solar dish', 'sombrero', 'soup bowl', 'space bar', 'space heater', 'space shuttle', 'spatula', 'speedboat', 'spider web', 'spindle', 'sports car', 'spotlight', 'stage', 'steam locomotive', 'steel arch bridge', 'steel drum', 'stethoscope', 'stole', 'stone wall', 'stopwatch', 'stove', 'strainer', 'streetcar', 'stretcher', 'studio couch', 'stupa', 'submarine', 'suit', 'sundial', 'sunglass', 'sunglasses', 'sunscreen', 'suspension bridge', 'swab', 'sweatshirt', 'swimming trunks', 'swing', 'switch', 'syringe', 'table lamp', 'tank', 'tape player', 'teapot', 'teddy', 'television', 'tennis ball', 'thatch', 'theater curtain', 'thimble', 'thresher', 'throne', 'tile roof', 'toaster', 'tobacco shop', 'toilet seat', 'torch', 'totem pole', 'tow truck', 'toyshop', 'tractor', 'trailer truck', 'tray', 'trench coat', 'tricycle', 'trimaran', 'tripod', 'triumphal arch', 'trolleybus', 'trombone', 'tub', 'turnstile', 'typewriter keyboard', 'umbrella', 'unicycle', 'upright', 'vacuum', 'vase', 'vault', 'velvet', 'vending machine', 'vestment', 'viaduct', 'violin', 'volleyball', 'waffle iron', 'wall clock', 'wallet', 'wardrobe', 'warplane', 'washbasin', 'washer', 'water bottle', 'water jug', 'water tower', 'whiskey jug', 'whistle', 'wig', 'window screen', 'window shade', 'Windsor tie', 'wine bottle', 'wing', 'wok', 'wooden spoon', 'wool', 'worm fence', 'wreck', 'yaw', 'yurt', 'web site', 'comic book', 'crossword puzzle', 'street sign', 'traffic light', 'book jacket', 'menu', 'plate', 'guacamole', 'consomme', 'hot pot', 'trifle', 'ice cream', 'ice lolly', 'French loaf', 'bagel', 'pretzel', 'cheeseburger', 'hotdog', 'mashed potato', 'head cabbage', 'broccoli', 'cauliflower', 'zucchini', 'spaghetti squash', 'acorn squash', 'butternut squash', 'cucumber', 'artichoke', 'bell pepper', 'cardoon', 'mushroom', 'Granny Smith', 'strawberry', 'orange', 'lemon', 'fig', 'pineapple', 'banana', 'jackfruit', 'custard apple', 'pomegranate', 'hay', 'carbonara', 'chocolate sauce', 'dough', 'meat loaf', 'pizza', 'potpie', 'burrito', 'red wine', 'espresso', 'cup', 'eggnog', 'alp', 'bubble', 'cliff', 'coral reef', 'geyser', 'lakeside', 'promontory', 'sandbar', 'seashore', 'valley', 'volcano', 'ballplayer', 'groom', 'scuba diver', 'rapeseed', 'daisy', 'yellow lady's slipper', 'corn', 'acorn', 'hip', 'buckeye', 'coral fungus', 'agaric', 'gyromitra', 'stinkhorn', 'earthstar', 'hen-of-the-woods', 'bolete', 'ear', 'toilet tissue']

```

recipe: https://github.com/pytorch/vision/pull/116
_docs: These weights are ported from LuaTorch.
num_params: 7978856
_metrics: {'ImageNet-1K': {'acc@1': 74.434, 'acc@5': 91.972}}
_ops: 2.834
_file_size: 30.845

```

```

[65]: # Looking at the pre training transforms
weights.transforms()

```

```

[65]: ImageClassification(
    crop_size=[224]
    resize_size=[256]
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    interpolation=InterpolationMode.BILINEAR
)

```

```

[66]: # Looking at the classifier head
model.classifier

```

```

[66]: Linear(in_features=1024, out_features=1000, bias=True)

```

```

[67]: # Updating the classifier head
NUM_CLASSES = 20
model.classifier = nn.Sequential(
    nn.Linear(model.classifier.in_features, 512),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(512, NUM_CLASSES)
)

```

```

[70]: # Parameter count
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Parameter Count:")
print(f"Total      : {total_params:,}")
print(f"Trainable   : {trainable_params:,}")

```

```

Parameter Count:
Total      : 7,488,916
Trainable   : 7,488,916

```

```

[68]: # Wrap model for multi-GPU after moving to device
if torch.cuda.device_count() > 1:
    print(f"Using {torch.cuda.device_count()} GPUs via DataParallel")
    model = nn.DataParallel(model)
else:

```

```
print("Single GPU or CPU - DataParallel not applied")

model = model.to(device)
```

Single GPU or CPU - DataParallel not applied

5 Define loss and optimizer

```
[69]: criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

Competition scoring function

```
[71]: def competition_score(true_labels, predicted_labels, num_classes=20):
    """
    TP = +1, FP = -1, FN = -5
    Returns macro-averaged score across all classes
    """
    scores = []
    for c in range(num_classes):
        tp = ((predicted_labels == c) & (true_labels == c)).sum()
        fp = ((predicted_labels == c) & (true_labels != c)).sum()
        fn = ((predicted_labels != c) & (true_labels == c)).sum()
        nc = (true_labels == c).sum()
        if nc == 0:
            continue
        score_c = (tp - fp - 5 * fn) / nc
        scores.append(score_c)
    return round(float(np.mean(scores)), 4)
```

6 Model training and evaluation

```
[ ]: # Constants
DEBUG_BATCHES = 5
TRAIN_PRINT_EVERY = 50
EVAL_PRINT_EVERY = 50
NUM_EPOCHS = 2 if DEBUG else 20
PATIENCE = 5
CHECKPOINT_PATH = Path('models/best_model.pth')
```

```
[75]: def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss = 0
    all_preds, all_labels = [], []

    for batch_idx, (images, labels) in enumerate(loader):
```

```

if DEBUG and batch_idx >= DEBUG_BATCHES:
    break

images = images.to(device)
labels = labels.to(device)

optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()

total_loss += loss.item()
preds = outputs.argmax(dim=1)
all_preds.extend(preds.cpu().numpy())
all_labels.extend(labels.cpu().numpy())

print_every = 1 if DEBUG else TRAIN_PRINT_EVERY
if batch_idx % print_every == 0:
    print(f" Batch {batch_idx}/{len(loader)} loss: {loss.item():.4f}")

num_batches = min(DEBUG_BATCHES, len(loader)) if DEBUG else len(loader)
avg_loss = total_loss / num_batches
return avg_loss, np.array(all_preds), np.array(all_labels)

```

```

[76]: def evaluate(model, loader):
    model.eval()
    all_preds, all_labels = [], []

    with torch.no_grad():
        for batch_idx, (images, labels) in enumerate(loader):
            if DEBUG and batch_idx >= DEBUG_BATCHES:
                break

            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            preds = outputs.argmax(dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

            print_every = 1 if DEBUG else EVAL_PRINT_EVERY
            if batch_idx % print_every == 0:
                print(f" Eval batch {batch_idx}/{len(loader)}")

    return np.array(all_preds), np.array(all_labels)

```

```

[77]: def run_training(model, train_loader, val_loader, criterion, optimizer):
    CHECKPOINT_PATH.parent.mkdir(parents=True, exist_ok=True)

    best_val_score = float('-inf')
    epochs_without_improvement = 0

    for epoch in range(NUM_EPOCHS):
        print(f"--- Epoch {epoch+1}/{NUM_EPOCHS} ---")

        print(" [Training...]")
        train_loss, tr_preds, tr_labels = train_one_epoch(model, train_loader,
↪criterion, optimizer)
        print(" [Training done]")

        print(" [Evaluating...]")
        val_preds, val_labels = evaluate(model, val_loader)
        print(" [Evaluation done]")

        train_score = competition_score(tr_labels, tr_preds)
        val_score = competition_score(val_labels, val_preds)
        print(f" Train loss: {train_loss:.4f} | Train score: {train_score} | Val
↪score: {val_score}\n")

        if val_score > best_val_score:
            best_val_score = val_score
            epochs_without_improvement = 0
            torch.save(model.state_dict(), CHECKPOINT_PATH)
            print(f" New best val score: {best_val_score:.4f} - model saved to
↪{CHECKPOINT_PATH}\n")
        else:
            epochs_without_improvement += 1
            print(f" No improvement for {epochs_without_improvement}/{PATIENCE}
↪epochs\n")
            if epochs_without_improvement >= PATIENCE:
                print(f"Early stopping triggered after {epoch+1} epochs.")
                break

    print(f"Training complete. Best val score: {best_val_score:.4f}")

```